ProjExD_4 / musou_kokaton.py

c0a2515564 残機修正 8103d13 · 2 minutes ago

Code Blame 529 lines (454 loc) · 19 KB Raw

```python
import math
import os
import random
import sys
import time
import pygame as pg


WIDTH = 1100  # ゲームウィンドウの幅
HEIGHT = 650  # ゲームウィンドウの高さ
os.chdir(os.path.dirname(os.path.abspath(__file__)))


def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
    """
    オブジェクトが画面内or画面外を判定し，真理値タプルを返す関数
    引数：こうかとんや爆弾，ビームなどのRect
    戻り値：横方向，縦方向のはみ出し判定結果（画面内：True／画面外：False）
    """
    yoko, tate = True, True
    if obj_rct.left < 0 or WIDTH < obj_rct.right:
        yoko = False
    if obj_rct.top < 0 or HEIGHT < obj_rct.bottom:
        tate = False
    return yoko, tate


def calc_orientation(org: pg.Rect, dst: pg.Rect) -> tuple[float, float]:
    """
    orgから見て，dstがどこにあるかを計算し，方向ベクトルをタプルで返す
    引数1 org：爆弾SurfaceのRect
    引数2 dst：こうかとんSurfaceのRect
    戻り値：orgから見たdstの方向ベクトルを表すタプル
    """
    x_diff, y_diff = dst.centerx-org.centerx, dst.centery-org.centery
    norm = math.sqrt(x_diff**2+y_diff**2)
    return x_diff/norm, y_diff/norm


class Bird(pg.sprite.Sprite):
    """
```

```python
    ｹﾞｰﾑｷｬﾗｸﾀｰ（こうかとん）に関するクラス
    delta = {  # 押下キーと移動量の辞書
        pg.K_UP: (0, -1),
        pg.K_DOWN: (0, +1),
        pg.K_LEFT: (-1, 0),
        pg.K_RIGHT: (+1, 0),
    }

    def __init__(self, num: int, xy: tuple[int, int]):
        """
        こうかとん画像Surfaceを生成する
        引数1 num：こうかとん画像ファイル名の番号
        引数2 xy：こうかとん画像の位置座標タプル
        """
        super().__init__()
        img0 = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
        img = pg.transform.flip(img0, True, False)  # デフォルトのこうかとん
        self.imgs = {
            (+1, 0): img,  # 右
            (+1, -1): pg.transform.rotozoom(img, 45, 0.9),  # 右上
            (0, -1): pg.transform.rotozoom(img, 90, 0.9),  # 上
            (-1, -1): pg.transform.rotozoom(img0, -45, 0.9),  # 左上
            (-1, 0): img0,  # 左
            (-1, +1): pg.transform.rotozoom(img0, 45, 0.9),  # 左下
            (0, +1): pg.transform.rotozoom(img, -90, 0.9),  # 下
            (+1, +1): pg.transform.rotozoom(img, -45, 0.9),  # 右下
        }
        self.dire = (+1, 0)
        self.image = self.imgs[self.dire]
        self.rect = self.image.get_rect()
        self.rect.center = xy
        self.speed = 10
        self.state = "normal" #ノーマルある状態(ここを変更して無敵か普通か変更する)
        self.hyper_life = 0  #無敵状態があと何フレーム続くか

    def change_img(self, num: int, screen: pg.Surface):
        """
        こうかとん画像を切り替え，画面に転送する
        引数1 num：こうかとん画像ファイル名の番号
        引数2 screen：画面Surface
        """
        self.image = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
        screen.blit(self.image, self.rect)

    def update(self, key_lst: list[bool], screen: pg.Surface):
        """
        押下キーに応じてこうかとんを移動させる
        引数1 key_lst：押下キーの真理値リスト
        引数2 screen：画面Surface
        """
        sum_mv = [0, 0]
        for k, mv in __class__.delta.items():
            if key_lst[k]:
                sum_mv[0] += mv[0]
                sum_mv[1] += mv[1]

        if self.state == "hyper": #無敵状態の時の処理
            self.hyper_life -= 1 #無敵時間を減らす
```

```python
                if self.hyper_life < 0: #無敵時間が0より下になったら
                    self.state = "normal"   #普通に戻す

            self.rect.move_ip(self.speed*sum_mv[0], self.speed*sum_mv[1])
            if check_bound(self.rect) != (True, True):
                self.rect.move_ip(-self.speed*sum_mv[0], -self.speed*sum_mv[1])
            if not (sum_mv[0] == 0 and sum_mv[1] == 0):
                self.dire = tuple(sum_mv)
                if self.state == "hyper":
                    self.image = self.imgs[self.dire].copy()
                    self.image = pg.transform.laplacian(self.image) #画像を変える
                else:
                    self.image = self.imgs[self.dire]
            screen.blit(self.image, self.rect)


class Bomb(pg.sprite.Sprite):
    """
    爆弾に関するクラス
    """
    colors = [(255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (255, 0, 255), (0, 255, 255)]

    def __init__(self, emy: "Enemy", bird: Bird):
        """
        爆弾円Surfaceを生成する
        引数1 emy：爆弾を投下する敵機
        引数2 bird：攻撃対象のこうかとん
        """
        super().__init__()
        rad = random.randint(10, 50)  # 爆弾円の半径：10以上50以下の乱数
        self.image = pg.Surface((2*rad, 2*rad))
        color = random.choice(__class__.colors)  # 爆弾円の色：クラス変数からランダム選択
        pg.draw.circle(self.image, color, (rad, rad), rad)
        self.image.set_colorkey((0, 0, 0))
        self.rect = self.image.get_rect()
        # 爆弾を投下するemyから見た攻撃対象のbirdの方向を計算
        self.vx, self.vy = calc_orientation(emy.rect, bird.rect)
        self.rect.centerx = emy.rect.centerx
        self.rect.centery = emy.rect.centery+emy.rect.height//2
        self.speed = 6
        self.state = True

    def update(self):
        """
        爆弾を速度ベクトルself.vx, self.vyに基づき移動させる
        引数 screen：画面Surface
        """
        self.rect.move_ip(self.speed*self.vx, self.speed*self.vy)
        if check_bound(self.rect) != (True, True):
            self.kill()


class Beam(pg.sprite.Sprite):
    """
    ビームに関するクラス
    """
    def __init__(self, bird: Bird, angle0: int = 0):
        """
```

```
159             ビーム画像Surfaceを生成する
160             引数 bird：ビームを放つこうかとん
161             """
162             super().__init__()
163             self.vx, self.vy = bird.dire
164             angle = math.degrees(math.atan2(-self.vy, self.vx)) + angle0
165             self.image = pg.transform.rotozoom(pg.image.load(f"fig/beam.png"), angle, 1.0)
166             self.vx = math.cos(math.radians(angle))
167             self.vy = -math.sin(math.radians(angle))
168             self.rect = self.image.get_rect()
169             self.rect.centery = bird.rect.centery+bird.rect.height*self.vy
170             self.rect.centerx = bird.rect.centerx+bird.rect.width*self.vx
171             self.speed = 10
172
173         def update(self):
174             """
175             ビームを速度ベクトルself.vx, self.vyに基づき移動させる
176             引数 screen：画面Surface
177             """
178             self.rect.move_ip(self.speed*self.vx, self.speed*self.vy)
179             if check_bound(self.rect) != (True, True):
180                 self.kill()
181
182     class NeoBeam():
183         """
184         弾幕生成クラス
185         """
186
187         def __init__(self, bird: Bird, num: int):
188             self.bird = bird
189             self.num = num
190
191
192         def gen_beams(self):
193             """
194             指定本数のビームを生成して返す
195             """
196             beams = []
197
198             if self.num == 1:
199                 beams.append(Beam(self.bird))
200                 return beams
201
202             for angle in range(-50, +50, 100//self.num):
203                 beams.append(Beam(self.bird, angle))
204
205             return beams
206
207     class Explosion(pg.sprite.Sprite):
208         """
209         爆発に関するクラス
210         """
211         def __init__(self, obj: "Bomb|Enemy", life: int):
212             """
213             爆弾が爆発するエフェクトを生成する
214             引数1 obj：爆発するBombまたは敵機インスタンス
215             引数2 life：爆発時間
216             """
```

```
217             super().__init__()
218             img = pg.image.load(f"fig/explosion.gif")
219             self.imgs = [img, pg.transform.flip(img, 1, 1)]
220             self.image = self.imgs[0]
221             self.rect = self.image.get_rect(center=obj.rect.center)
222             self.life = life
223 
224         def update(self):
225             """
226             爆発時間を1減算した爆発経過時間_lifeに応じて爆発画像を切り替えることで
227             爆発エフェクトを表現する
228             """
229             self.life -= 1
230             self.image = self.imgs[self.life//10%2]
231             if self.life < 0:
232                 self.kill()
233 
234 
235     class Enemy(pg.sprite.Sprite):
236         """
237         敵機に関するクラス
238         """
239         imgs = [pg.image.load(f"fig/alien{i}.png") for i in range(1, 4)]
240 
241         def __init__(self):
242             super().__init__()
243             self.image = pg.transform.rotozoom(random.choice(__class__.imgs), 0, 0.8)
244             self.rect = self.image.get_rect()
245             self.rect.center = random.randint(0, WIDTH), 0
246             self.vx, self.vy = 0, +6
247             self.bound = random.randint(50, HEIGHT//2)  # 停止位置
248             self.state = "down"  # 降下状態or停止状態
249             self.interval = random.randint(50, 300)  # 爆弾投下インターバル
250 
251         def update(self):
252             """
253             敵機を速度ベクトルself.vyに基づき移動（降下）させる
254             ランダムに決めた停止位置_boundまで降下したら，_stateを停止状態に変更する
255             引数 screen：画面Surface
256             """
257             if self.rect.centery > self.bound:
258                 self.vy = 0
259                 self.state = "stop"
260             self.rect.move_ip(self.vx, self.vy)
261 
262 
263     class Emp:
264         """EMPに関するクラス"""
265         def __init__(self, enemys: pg.sprite.Group, bombs: pg.sprite.Group, screen: pg.Surface, life: int = 3):
266             """
267             起動時に表示する画像を生成する。
268             引数1 enemys: 停止される敵group
269             引数2 bombs: 停止される爆弾group
270             引数3 screen: 画面Surface
271             引数4 life: 起動時の画面表示時間
272             """
273             self.enemys = enemys
274             self.bombs = bombs
```

```
275             self.screen = screen
276             self.life = life
277             self.cost = 20
278             self.image = pg.Surface((WIDTH, HEIGHT))
279             pg.draw.rect(self.image, (255, 255, 0), (0, 0, WIDTH, HEIGHT))
280             self.image.set_alpha(64)
281 
282         def on(self):
283             """
284             EMPを起動する
285             """
286             for enemy in self.enemys:
287                 enemy.interval = math.inf
288                 enemy.image = pg.transform.laplacian(enemy.image)
289 
290             for bomb in self.bombs:
291                 bomb.speed = bomb.speed // 2
292                 bomb.state = False
293 
294         def update(self):
295             """
296             起動時の黄色透明画面を表示する
297             """
298             if self.life < 0:
299                 return
300             self.life -= 1
301             self.screen.blit(self.image, self.image.get_rect())
302 
303 
304     class Life:
305         """
306         残機数が0になるまで死なない
307         初期残機数：3
308         発動条件：爆弾に当たるたびに1減る
309         描画位置：画面右下
310         """
311         def __init__(self, num: int):
312             self.num = num
313             self.image = pg.Surface((40, 40), pg.SRCALPHA)
314 
315             points = [(16*math.sin(t/100)**3 +20,
316                     -(13*math.cos(t/100)-5*math.cos(2*t/100)-2*math.cos(3*t/100)-math.cos(4*t/100)) +20
317                     ) for t in range(0, 628) ]
318             pg.draw.polygon(self.image, (255, 0, 0), points)
319 
320         def update(self, screen: pg.Surface):
321             for i in range(self.num):
322                 x = WIDTH - 50 - 20 - i * 45
323                 y = HEIGHT - 50 - 20
324                 screen.blit(self.image, (x, y))
325 
326 
327     class Score:
328         """
329         打ち落とした爆弾，敵機の数をスコアとして表示するクラス
330         爆弾：1点
331         敵機：10点
332         """
```

```
333         def __init__(self):
334             self.font = pg.font.Font(None, 50)
335             self.color = (0, 0, 255)
336             self.value = 10000
337             self.image = self.font.render(f"Score: {self.value}", 0, self.color)
338             self.rect = self.image.get_rect()
339             self.rect.center = 100, HEIGHT-50
340 
341         def update(self, screen: pg.Surface):
342             self.image = self.font.render(f"Score: {self.value}", 0, self.color)
343             screen.blit(self.image, self.rect)
344 
345 
346     class Gravity(pg.sprite.Sprite):
347         def __init__(self,life:int):
348             super().__init__()
349             self.image = pg.Surface((WIDTH,HEIGHT))
350             pg.draw.rect(self.image,(0,0,0),(0,0,WIDTH,HEIGHT))
351             self.image.set_alpha(128)
352             self.rect = self.image.get_rect()
353             self.life = life
354 
355         def update(self):
356             self.life -= 1
357             if self.life < 0:
358                 self.kill()
359 
360 
361     class Shield(pg.sprite.Sprite):
362         """
363         シールドに関するクラス
364         """
365         def __init__(self, bird: Bird, life: int):
366             super().__init__()
367             self.bird = bird
368             self.life = life
369             self.original_image = pg.Surface((20, 2 * bird.rect.height))
370             self.original_image.set_colorkey((0, 0, 0))
371             pg.draw.rect(self.original_image, (0, 0, 255), (0, 0, 20, 2 * bird.rect.height))
372             vx, vy = self.bird.dire
373             nrm = math.sqrt((vx**2 + vy**2))
374             self.image = pg.transform.rotate(self.original_image, math.degrees(math.atan2(-vy, vx)))
375             self.rect = self.image.get_rect()
376             self.rect.center = self.bird.rect.center
377             self.rect.move_ip(self.bird.rect.width * vx / nrm, self.bird.rect.width * vy / nrm)
378 
379         def update(self):
380             """
381             回転と位置調整をする
382             """
383             vx, vy = self.bird.dire
384             nrm = math.sqrt((vx**2 + vy**2))
385             self.image = pg.transform.rotate(self.original_image, math.degrees(math.atan2(-vy, vx)))
386             self.rect = self.image.get_rect()
387             self.rect.center = self.bird.rect.center
388             self.rect.move_ip(self.bird.rect.width * vx / nrm, self.bird.rect.width * vy / nrm)
389             self.life -= 1
390             if self.life < 0:
391                 self.kill()
```

```python


def main():
    pg.display.set_caption("真！こうかとん無双")
    screen = pg.display.set_mode((WIDTH, HEIGHT))
    bg_img = pg.image.load(f"fig/pg_bg.jpg")
    score = Score()

    life = Life(3)

    bird = Bird(3, (900, 400))
    bombs = pg.sprite.Group()
    beams = pg.sprite.Group()
    exps = pg.sprite.Group()
    emys = pg.sprite.Group()
    emp = Emp(emys, bombs, screen, 0)
    gras = pg.sprite.Group()
    shields = pg.sprite.Group()
    nbeam = NeoBeam(bird, 5)

    tmr = 0
    clock = pg.time.Clock()
    while True:
        key_lst = pg.key.get_pressed()

        for event in pg.event.get():
            if event.type == pg.QUIT:
                return 0
            if event.type == pg.KEYDOWN and key_lst[pg.K_LSHIFT] and event.key == pg.K_SPACE :
                beams.add(nbeam.gen_beams())
            elif event.type == pg.KEYDOWN and event.key == pg.K_SPACE:
                beams.add(Beam(bird))
            if event.type == pg.KEYDOWN and event.key == pg.K_e and score.value >= emp.cost:
                score.value -= emp.cost
                emp = Emp(emys, bombs, screen)
                emp.on()

            if event.type == pg.KEYDOWN and event.key == pg.K_RETURN:
                # 重力場の発動
                if score.value > 200:
                    gras.add(Gravity(400))
                    score.value -= 200

            #無敵ここから
            if   event.type == pg.KEYDOWN and event.key == pg. K_RSHIFT: #右のシフトキーを押したら
                if bird.state == "normal" and score.value > 100:
                    bird.state = "hyper"
                    bird.hyper_life = 500
                    score.value -= 100
                    beams.add(Beam(bird))

            if event.type == pg.KEYDOWN and event.key == pg.K_s and score.value > 50 and len(shields) == 0:
                score.value -= 50
                shields.add(Shield(bird, 400))

        screen.blit(bg_img, [0, 0])

        if tmr%200 == 0:  # 200フレームに1回，敵機を出現させる
```

```python
            emys.add(Enemy())

        for emy in emys:
            if emy.state == "stop" and tmr%emy.interval == 0:
                # 敵機が停止状態に入ったら，intervalに応じて爆弾投下
                bombs.add(Bomb(emy, bird))

        for emy in pg.sprite.groupcollide(emys, beams, True, True).keys():  # ビームと衝突した敵機リスト
            exps.add(Explosion(emy, 100))  # 爆発エフェクト
            score.value += 10  # 10点アップ
            bird.change_img(6, screen)  # こうかとん喜びエフェクト

        if bird.state == "hyper":
            for emy in pg.sprite.spritecollide(bird, emys, True):
                exps.add(Explosion(emy, 100))
                score.value += 10

        for bomb in pg.sprite.groupcollide(bombs, beams, True, True).keys():  # ビームと衝突した爆弾リスト
            exps.add(Explosion(bomb, 50))  # 爆発エフェクト
            score.value += 1  # 1点アップ

        for emy in pg.sprite.groupcollide(emys, gras, True, False).keys():  # ビームと衝突した敵機リスト
            exps.add(Explosion(emy, 100))  # 爆発エフェクト
            score.value += 10   # 10点アップ
            bird.change_img(6, screen)  # こうかとん喜びエフェクト

        for bomb in pg.sprite.groupcollide(bombs, gras, True, False).keys():  # ビームと衝突した爆弾リスト
            exps.add(Explosion(bomb, 50))  # 爆発エフェクト
            score.value += 1  # 1点アップ

        for bomb in pg.sprite.groupcollide(bombs, shields, True, False).keys():  # シールドと衝突した爆弾リス
            exps.add(Explosion(bomb, 50))  # 爆発エフェクト

        for bomb in pg.sprite.spritecollide(bird, bombs, True):  # こうかとんと衝突した爆弾リスト
            if bird.state == "hyper":
                exps.add(Explosion(bomb, 50))
                score.value += 1
            elif bomb.state:
                life.num -= 1
                life.update(screen)
                score.update(screen)
                if life.num <= 0: # 0になったら終了
                    bird.change_img(8, screen)  # こうかとん悲しみエフェクト
                    life.update(screen)
                    score.update(screen)
                    pg.display.update()
                    time.sleep(1) # 1秒待機にしている
                    return

                bird.image = bird.imgs[bird.dire] # 元に戻す
            else:
                pass


        bird.update(key_lst, screen)
        beams.update()
        beams.draw(screen)
        emys.update()
```

```
            emys.draw(screen)
            bombs.update()
            bombs.draw(screen)
            exps.update()
            exps.draw(screen)
            gras.update()
            gras.draw(screen)
            shields.update()
            shields.draw(screen)
            score.update(screen)
            emp.update()
            life.update(screen)
            pg.display.update()
            tmr += 1
            clock.tick(50)


    if __name__ == "__main__":
        pg.init()
        main()
        pg.quit()
        sys.exit()
```